

a. Name of two GenAI programs

The 2 Ai that I used for this assignment was Gemini and Chat GPT.

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed both of these AIs through Google query on my laptop, in which I used the main websites for both AIs.

c. Prompt used. (You must use the same prompt for both GenAIs.)

1. Use two GenAI's to generate C++ code using objects, not functions, to implement

the following requirements, which are the same as Assignment 2

(Note: You may

use either ChatGPT or CoPilot as one of your choices, but not both.

They are

essentially the same GenAI.):

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A

priority queue is a queue where emails can shift towards the front of the queue

based on a priority status.

- You must implement a MaxHeap using a list-based implementation.

Then use

that MaxHeap to handle all your email prioritizing for the CEO.

- You must create objects from scratch. Do not include pre-existing heap modules.

- Test File format:

Command Description

EMAIL <sender category>,

<subject line>,

<date>

The emails in the CEO's Inbox should be placed in queue

based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin

with the emails in the CEOs Inbox, followed by the commands the CEO will

use to read her Inbox. You can assume a given command will be formatted as

shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

d. Generated code from both GenAIs.

Gemini Code:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <stdexcept>
```

```
// =====
```

```
// Class: Email
```

```
// Represents an email message with priority logic
```

```
// =====
```

```
class Email {
```

```
private:
```

```
    std::string senderCategory;
```

```
    std::string subjectLine;
```

```
    std::string dateStr; // Format: MM-DD-YYYY
```

```
    // Assign numeric ranking to sender categories:
```

```
    // Boss (5) > Subordinate (4) > Peer (3) > ImportantPerson (2) >
    OtherPerson (1)
```

```
    int getCategoryRank() const {
```

```
        if (senderCategory == "Boss") return 5;
```

```
        if (senderCategory == "Subordinate") return 4;
```

```
        if (senderCategory == "Peer") return 3;
```

```
        if (senderCategory == "ImportantPerson") return 2;
```

```
        if (senderCategory == "OtherPerson") return 1;
```

```
        return 0;
```

```
    }
```

```
// Convert MM-DD-YYYY to an integer YYYYMMDD for  
chronological comparison
```

```
int getNormalizedDate() const {  
    if (dateStr.length() < 10) return 0;  
    int month = std::stoi(dateStr.substr(0, 2));  
    int day = std::stoi(dateStr.substr(3, 2));  
    int year = std::stoi(dateStr.substr(6, 4));  
    return year * 10000 + month * 100 + day;  
}
```

```
public:
```

```
    Email() : senderCategory(""), subjectLine(""), dateStr("") {}
```

```
    Email(const std::string& sender, const std::string& subject, const  
std::string& date)
```

```
        : senderCategory(sender), subjectLine(subject), dateStr(date) {}
```

```
const std::string& getSender() const { return senderCategory; }
```

```
const std::string& getSubject() const { return subjectLine; }
```

```
const std::string& getDate() const { return dateStr; }
```

```
// Priority ordering: Higher rank first.
```

```
// If rank is equal, newer date (larger YYYYMMDD) has higher  
priority.
```

```
bool operator<(const Email& other) const {  
    int myRank = getCategoryRank();  
    int otherRank = other.getCategoryRank();
```

```
    if (myRank != otherRank) {  
        return myRank < otherRank;  
    }
```

```

        return getNormalizedDate() < other.getNormalizedDate();
    }

    bool operator>(const Email& other) const {
        return other < *this;
    }

    void display() const {
        std::cout << "Sender: " << senderCategory << "\n";
        std::cout << "Subject: " << subjectLine << "\n";
        std::cout << "Date: " << dateStr << "\n";
    }
};

// =====
// Class Template: MaxHeap
// List-based implementation created from scratch
// =====
template <typename T>
class MaxHeap {
private:
    std::vector<T> heapList;

    int parent(int index) const { return (index - 1) / 2; }
    int leftChild(int index) const { return 2 * index + 1; }
    int rightChild(int index) const { return 2 * index + 2; }

    void siftUp(int index) {
        while (index > 0 && heapList[index] > heapList[parent(index)]) {
            std::swap(heapList[index], heapList[parent(index)]);
            index = parent(index);
        }
    }
};

```

```
}
```

```
void siftDown(int index) {  
    int maxIndex = index;  
    int left = leftChild(index);  
    int right = rightChild(index);  
    int n = static_cast<int>(heapList.size());  
  
    if (left < n && heapList[left] > heapList[maxIndex]) {  
        maxIndex = left;  
    }  
  
    if (right < n && heapList[right] > heapList[maxIndex]) {  
        maxIndex = right;  
    }  
  
    if (index != maxIndex) {  
        std::swap(heapList[index], heapList[maxIndex]);  
        siftDown(maxIndex);  
    }  
}
```

```
public:
```

```
    MaxHeap() = default;
```

```
void insert(const T& item) {  
    heapList.push_back(item);  
    siftUp(static_cast<int>(heapList.size()) - 1);  
}
```

```
T extractMax() {  
    if (heapList.empty()) {
```

```

        throw std::underflow_error("Heap is empty.");
    }
    T maxValue = heapList[0];
    heapList[0] = heapList.back();
    heapList.pop_back();

    if (!heapList.empty()) {
        siftDown(0);
    }
    return maxValue;
}

const T& peekMax() const {
    if (heapList.empty()) {
        throw std::underflow_error("Heap is empty.");
    }
    return heapList[0];
}

bool isEmpty() const {
    return heapList.empty();
}

int size() const {
    return static_cast<int>(heapList.size());
}
};

// =====
// Class: CEOInboxManager
// Orchestrates commands and queue handling
// =====

```

```

class CEOInboxManager {
private:
    MaxHeap<Email> inboxQueue;

    // Helper method to trim leading and trailing whitespace
    static std::string trim(const std::string& str) {
        size_t first = str.find_first_not_of(" \t\r\n");
        if (first == std::string::npos) return "";
        size_t last = str.find_last_not_of(" \t\r\n");
        return str.substr(first, (last - first + 1));
    }

public:
    CEOInboxManager() = default;

    void addEmail(const std::string& sender, const std::string& subject,
const std::string& date) {
        Email email(trim(sender), trim(subject), trim(date));
        inboxQueue.insert(email);
    }

    void handleNext() const {
        if (inboxQueue.isEmpty()) {
            std::cout << "No emails in inbox.\n";
            return;
        }
        std::cout << "Next email:\n";
        inboxQueue.peekMax().display();
    }

    void handleRead() {
        if (inboxQueue.isEmpty()) {

```

```

        std::cout << "No emails to read.\n";
        return;
    }
    inboxQueue.extractMax();
}

void handleCount() const {
    std::cout << "There are " << inboxQueue.size() << " emails to
read.\n";
}

void processCommandFile(const std::string& filename) {
    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Error: Could not open file " << filename << "\n";
        return;
    }

    std::string line;
    while (std::getline(file, line)) {
        line = trim(line);
        if (line.empty()) continue;

        if (line.rfind("EMAIL ", 0) == 0) {
            // Remove the "EMAIL " command prefix
            std::string payload = line.substr(6);
            std::stringstream ss(payload);
            std::string sender, subject, date;

            if (std::getline(ss, sender, ',') &&
                std::getline(ss, subject, ',') &&
                std::getline(ss, date)) {

```

```

        addEmail(sender, subject, date);
    }
    } else if (line == "NEXT") {
        handleNext();
    } else if (line == "READ") {
        handleRead();
    } else if (line == "COUNT") {
        handleCount();
    }
    }
    file.close();
}
};

// =====
// Main Entry Point
// =====
int main(int argc, char* argv[]) {
    std::string filename = "test.txt";
    if (argc > 1) {
        filename = argv[1];
    }

    CEOInboxManager manager;
    manager.processCommandFile(filename);

    return 0;
}

```

Open AI Code:

```

#include <iostream>
#include <fstream>

```

```
#include <string>
#include <sstream>
```

```
using namespace std;
```

```
//
=====

=====
// EMAIL CLASS
//
=====

=====
// This object represents one email in the CEO's inbox.
//
=====

=====
class Email
{
private:
    string senderCategory;
    string subject;
    string date;

public:

    // Constructor
    Email()
    {
        senderCategory = "";
        subject = "";
        date = "";
    }
}
```

```
}
```

```
// Constructor with email information
```

```
Email(string sender, string sub, string d)
```

```
{
```

```
    senderCategory = sender;
```

```
    subject = sub;
```

```
    date = d;
```

```
}
```

```
// Return the sender category
```

```
string getSenderCategory()
```

```
{
```

```
    return senderCategory;
```

```
}
```

```
// Return the subject
```

```
string getSubject()
```

```
{
```

```
    return subject;
```

```
}
```

```
// Return the date
```

```
string getDate()
```

```
{
```

```
    return date;
```

```
}
```

```
// Display the email in the required format
```

```
void display()
```

```
{
```

```
    cout << "Sender: " << senderCategory << endl;
```

```

        cout << "Subject: " << subject << endl;
        cout << "Date: " << date << endl;
    }
};

//
=====

=====
// MAXHEAP CLASS
//
=====

=====
// This class implements a MaxHeap from scratch.
//
// The heap is list/array based.
// No C++ priority_queue or other heap module is used.
//
=====

=====
class MaxHeap
{
private:

    // Dynamically allocated array used as our list
    Email* heap;

    // Number of emails currently in the heap
    int heapSize;

    // Maximum number of emails the heap can hold
    int capacity;

```

```
// -----  
// Convert a sender category into a priority number.  
//  
// Higher number = higher priority.  
// -----  
int getCategoryPriority(string category)  
{  
    if (category == "Boss")  
    {  
        return 5;  
    }  
    else if (category == "Subordinate")  
    {  
        return 4;  
    }  
    else if (category == "Peer")  
    {  
        return 3;  
    }  
    else if (category == "ImportantPerson")  
    {  
        return 2;  
    }  
    else  
    {  
        return 1; // OtherPerson  
    }  
}
```

```

// -----
// Convert MM-DD-YYYY into an integer.
//
// Example:
// 01-03-2025 becomes 20250103
//
// This allows dates to be compared numerically.
// -----
int convertDate(string date)
{
    int month;
    int day;
    int year;

    char dash1;
    char dash2;

    stringstream ss(date);

    ss >> month >> dash1 >> day >> dash2 >> year;

    return year * 10000 + month * 100 + day;
}

// -----
// Determines whether email1 has a higher priority
// than email2.
//
// Priority rules:
// 1. Sender category
// 2. Newest date if sender categories are equal

```

```
// -----  
bool higherPriority(Email email1, Email email2)  
{  
    int priority1 =  
        getCategoryPriority(email1.getSenderCategory());  
  
    int priority2 =  
        getCategoryPriority(email2.getSenderCategory());  
  
    // First compare sender categories  
    if (priority1 > priority2)  
    {  
        return true;  
    }  
  
    if (priority1 < priority2)  
    {  
        return false;  
    }  
  
    // If categories are the same,  
    // compare dates.  
    int date1 = convertDate(email1.getDate());  
    int date2 = convertDate(email2.getDate());  
  
    return date1 > date2;  
}  
  
// -----
```

```

// Swap two Email objects.
// -----
void swapEmails(Email& first, Email& second)
{
    Email temporary = first;

    first = second;

    second = temporary;
}

// -----
// Move an email upward through the heap.
// Used after inserting a new email.
// -----
void heapifyUp(int index)
{
    while (index > 0)
    {
        // Parent of index i is (i - 1) / 2
        int parent = (index - 1) / 2;

        // If the current email has greater priority
        // than its parent, swap them.
        if (higherPriority(heap[index], heap[parent]))
        {
            swapEmails(heap[index], heap[parent]);

            index = parent;
        }
    }
}

```

```

        else
        {
            break;
        }
    }
}

```

```

// -----
// Move an email downward through the heap.
// Used after removing the highest-priority email.
// -----
void heapifyDown(int index)
{
    while (true)
    {
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;

        int largest = index;

        // Check left child
        if (leftChild < heapSize &&
            higherPriority(heap[leftChild], heap[largest]))
        {
            largest = leftChild;
        }

        // Check right child
        if (rightChild < heapSize &&

```

```

        higherPriority(heap[rightChild], heap[largest]))
    {
        largest = rightChild;
    }

    // If index is already the highest-priority
    // position, the heap is finished.
    if (largest == index)
    {
        break;
    }

    // Otherwise swap with the highest-priority child.
    swapEmails(heap[index], heap[largest]);

    index = largest;
}
}

```

public:

```

// -----
// Constructor
// -----
MaxHeap(int initialCapacity = 100)
{
    capacity = initialCapacity;

    heapSize = 0;
}

```

```

    heap = new Email[capacity];
}

// -----
// Destructor
//
// Frees the dynamically allocated memory.
// -----
~MaxHeap()
{
    delete[] heap;
}

// -----
// Return the current number of emails.
// -----
int size()
{
    return heapSize;
}

// -----
// Determine whether the heap is empty.
// -----
bool isEmpty()
{
    return heapSize == 0;
}

```

```

// -----
// Add an email to the MaxHeap.
// -----
void insert(Email email)
{
    // If the array is full, create a larger array.
    if (heapSize == capacity)
    {
        int newCapacity = capacity * 2;

        Email* newHeap = new Email[newCapacity];

        // Copy existing emails into the larger array.
        for (int i = 0; i < heapSize; i++)
        {
            newHeap[i] = heap[i];
        }

        // Delete the old array.
        delete[] heap;

        // Point heap to the new array.
        heap = newHeap;

        capacity = newCapacity;
    }
}

```

```

    // Put the new email at the end of the heap.
    heap[heapSize] = email;

    // Increase the number of emails.
    heapSize++;

    // Move the email into its proper position.
    heapifyUp(heapSize - 1);
}

// -----
// Return the highest-priority email without removing it.
//
// This is what NEXT needs.
// -----
Email getMax()
{
    return heap[0];
}

// -----
// Remove the highest-priority email.
//
// This is what READ needs.
// -----
void removeMax()
{
    // Do nothing if there are no emails.

```

```

    if (heapSize == 0)
    {
        return;
    }

    // Move the last email to the root.
    heap[0] = heap[heapSize - 1];

    // Decrease the heap size.
    heapSize--;

    // Restore the MaxHeap property.
    if (heapSize > 0)
    {
        heapifyDown(0);
    }
}
};

//
=====

=====
// EMAIL MANAGER CLASS
//
=====

=====
// This class handles the input commands and uses the MaxHeap.

```

```
//
=====
=====
class EmailManager
{
private:

    MaxHeap emailHeap;

    // -----
    // Process an EMAIL command.
    //
    // Example:
    // EMAIL Boss,Important,12-20-2024
    // -----
    void processEmail(string line)
    {
        // Remove "EMAIL " from the beginning.
        string emailInformation = line.substr(6);

        // Find the first comma.
        size_t firstComma = emailInformation.find(',');

        // Find the second comma.
        size_t secondComma =
            emailInformation.find(',', firstComma + 1);

        // Extract sender category.
```

```

string sender =
    emailInformation.substr(0, firstComma);

// Extract subject.
string subject =
    emailInformation.substr(
        firstComma + 1,
        secondComma - firstComma - 1
    );

// Extract date.
string date =
    emailInformation.substr(secondComma + 1);

// Create an Email object.
Email email(sender, subject, date);

// Insert the email into the MaxHeap.
emailHeap.insert(email);
}

// -----
// Process NEXT.
//
// NEXT displays the highest-priority email but does NOT
// remove it.
// -----

```

```
void processNext()
{
    if (!emailHeap.isEmpty())
    {
        cout << "Next email:" << endl;

        Email nextEmail = emailHeap.getMax();

        nextEmail.display();
    }
}
```

```
// -----
// Process READ.
//
// READ removes the highest-priority email.
// It does not display the email.
// -----
```

```
void processRead()
{
    if (!emailHeap.isEmpty())
    {
        emailHeap.removeMax();
    }
}
```

```
// -----
// Process COUNT.
// -----
void processCount()
```

```
{  
    cout << "There are "  
        << emailHeap.size()  
        << " emails to read."  
        << endl;  
}
```

public:

```
// -----  
// Read and process the entire input file.  
// -----  
void processFile(string filename)  
{  
    ifstream inputFile(filename);  
  
    // Make sure the file opened successfully.  
    if (!inputFile)  
    {  
        cout << "Error: Could not open file."  
            << endl;  
  
        return;  
    }
```

```
    string line;
```

```
    // Read one line at a time.
```

```
while (getline(inputFile, line))
{
    // Ignore empty lines.
    if (line.empty())
    {
        continue;
    }

    // EMAIL command
    if (line.substr(0, 5) == "EMAIL")
    {
        processEmail(line);
    }

    // NEXT command
    else if (line == "NEXT")
    {
        processNext();
    }

    // READ command
    else if (line == "READ")
    {
        processRead();
    }

    // COUNT command
    else if (line == "COUNT")
```

```
    {  
        processCount();  
    }  
}
```

```
    // Close the input file.  
    inputFile.close();  
}  
};
```

```
//  
=====
```

=====

```
// MAIN  
//  
=====
```

=====

```
int main()  
{  
    // Create an EmailManager object.  
    EmailManager manager;  
  
    // Process the test file.  
    manager.processFile("test.txt");  
  
    return 0;  
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Gemini Code:

Correctness: The code seems to be able to only work when It has a .txt file that it takes in, rather than working either way with either a txt file or a manual input like the last assignment. Although the given prompt does work with the given text file. It also doesn't work if the commands are lowercase.

Execution Time:

Execution time for the code is approximately 500 ms. The time complexity of the code is $O(m \log n)$, which is estimated to be the theoretical optimum for comparison based priority queues.

Space Complexity: The general space complexity of this program is $O(n)$, while It also includes $O(1)$ footprint per email that is saved, the function sift_down is implemented recursively, which adds $O(\log n)$ stack frames during extractions.

Maintainability: The code is very well organized and is split up very well between the various functions, giving constants when needed and explaining what each part of the code does which makes the code much easier to read and understand.

Chat GPT Code:

Correctness: Executes with the .txt file and essentially works the same way as the Gemini code does. It also isn't able to take in user input for emails and only works as a way to take in file I/O.

Execution Time: Execution time seems to be significantly lower than the first program, averaging at around 450 ms and almost never going over 500 ms. Similarly, the code has a time complexity of $O(n \log n)$ just like the Gemini code is.

Space complexity: The space complexity of this program also turns out to be $O(n)$ due to the heap that holds all of the emails.

Maintainability: The code is significantly larger than the Gemini code, which makes it somewhat harder to understand since it's not as condensed as It could be and still run to the same efficiency.

f. Select one to use as the basis for this assignment and justify why you picked it.

I decided to use the Gemini code over the ChatGPT code for the final version because it's a lot more maintainable and legible than the one made by ChatGPT and I think It will be much easier to add in extra features and tweaks with the way that Gemini decided to structure the code.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: I implemented strict data parsing, because `std::stoi(dateStr.substr(...))` could produce `std::invalid_argument` or `std::out_of_range` if there's ever any unexpected lines or data slips in the input.

Execution time:

I decided to store `categoryRank` (int) and `normalizedDate` (int) as member variables in order to turn the 2 primitive integers comparisons $O(1)$ and make for zero allocations. I also eliminated recursion by swapping to a while loop in order to reduce execution time.

Space complexity:

I created an iterative `sift_down` method, which reduces the call stack memory from $O(\log n)$ to just $O(1)$. Additionally, I store an enum class `SenderCategory : uint8_t` for every Email node that is given.

Maintainability: I replaced the error-prone magic strings with enums, which should cause less issues in the code. Additionally, I created explicit default constructor methods and marked reference parameters and query methods by marking them with "const".